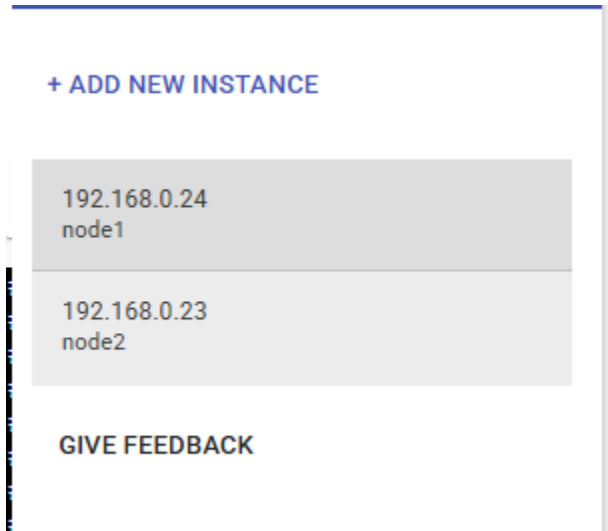


CO515: Advances in Computer Networks – Selected Topics

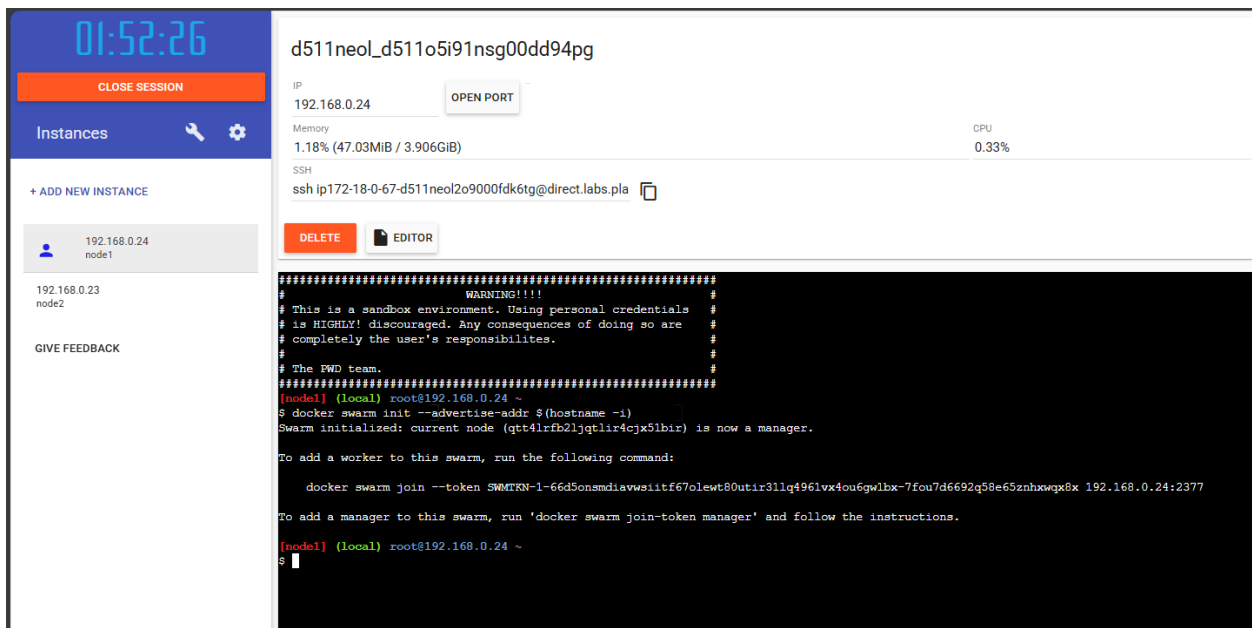
Lab 02 - In-Lab Assignment

Docker Overlay Networking - E20280

- Initialize two nodes



- Initialize swarm



Worker joined the swarm

```
[node2] (local) root@192.168.0.23 ~
$ docker swarm join --token SWMTKN-1-66d5onsmdiaovsijt67olewt80utir311q4961vx4ou6gw1bx-7fou7d6692q58e65znxwqx8x 192.168.0.24:2377
This node joined a swarm as a worker.
[node2] (local) root@192.168.0.23 ~
```

- verify nodes using ls (Screenshot 1)

```
[node1] (local) root@192.168.0.24 ~
$ docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER	STATUS	ENGINE	VERSION
qtt41rfb2ljqt1ir4cjx51bir *	node1	Ready	Active	Leader		27.3.1	
eckim4dtt8ggin8tvvkellz8f	node2	Ready	Active			27.3.1	

Step2:

- All networks on node 1

```
[node1] (local) root@192.168.0.24 ~
$ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
9a5a80d04908	bridge	bridge	local
57792a18a041	docker_gwbridge	bridge	local
b767b1fb0d05	host	host	local
w8k8bc3syjxg	ingress	overlay	swarm
ef29292b3973	none	null	local

- Inspect ingress network since swarm is in ingress

```
[node1] (local) root@192.168.0.24 ~
$ docker network inspect ingress
[
  {
    "Name": "ingress",
    "Id": "w8k8bc3syjxgc6lumrs73lo6y",
    "Created": "2025-12-17T03:03:22.083744203Z",
    "Scope": "swarm",
    "Driver": "overlay",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "10.0.0.0/24",
          "Gateway": "10.0.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": true,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "ingress-sbox": {
        "Name": "ingress-endpoint",
        "EndpointID": "63d1fbc7b19f452aff54983e5da99ce79713e5b9c04ab49c76946e7fc1e9db52",
        "MacAddress": "02:42:0a:00:00:02",
        "IPv4Address": "10.0.0.2/24",
        "IPv6Address": ""
      }
    },
    "Options": {
      "com.docker.network.driver.overlay.vxlanid_list": "4096"
    },
    "Labels": {},
    "Peers": [
      {
        "Name": "d08527359b3a",
        "IP": "192.168.0.24"
      },
      {
        "Name": "dd8b096f5b46",
        "IP": "192.168.0.23"
      }
    ]
  }
]
```

- All networks in node 2 (Screenshot 2)

```
[node2] (local) root@192.168.0.23 ~
$ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
075e23bd7409	bridge	bridge	local
6fc1819fdc3c	docker_gwbridge	bridge	local
3047c33dd453	host	host	local
w8k8bc3syjxg	ingress	overlay	swarm
a432d775a218	none	null	local

Step3:

- Create overlay network on node 1

```
[node1] (local) root@192.168.0.24 ~
$ docker network create -d overlay appnet
3o8yqs2lir6wbpgsbfmts2llp
```

- Verify if created the network

```
[node1] (local) root@192.168.0.24 ~
$ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
3o8yqs2lir6w	appnet	overlay	swarm
9a5a80d04908	bridge	bridge	local
57792a18a041	docker_gwbridge	bridge	local
b767b1fb0d05	host	host	local
w8k8bc3syjxg	ingress	overlay	swarm
ef29292b3973	none	null	local

- Inspect appnet network

```
[node1] (local) root@192.168.0.24 ~
$ docker network inspect appnet
[
  {
    "Name": "appnet",
    "Id": "3o8yqs2lir6wbpgsbfmts2llp",
    "Created": "2025-12-17T03:28:27.412147888Z",
    "Scope": "swarm",
    "Driver": "overlay",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "10.0.1.0/24",
          "Gateway": "10.0.1.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": null,
    "Options": {
      "com.docker.network.driver.overlay.vxlanid_list": "4097"
    },
    "Labels": null
  }
]
```

- Checking if node 2 able to see appnet network

```
[node2] (local) root@192.168.0.23 ~
$ docker network ls
NETWORK ID      NAME                DRIVER             SCOPE
075e23bd7409    bridge              bridge              local
6fc1819fdc3c    docker_gwbridge     bridge              local
3047c33dd453    host                host                local
w8k8bc3syjxg    ingress             overlay             swarm
a432d775a218    none                null                local
```

Not visible since no containers attached yet

Step4:

- Creating “backend” service with 4 replicas in node 1

```
[node1] (local) root@192.168.0.24 ~
$ docker service create \
  --name backend \
  --network appnet \
  --replicas 4 \
  alpine sleep infinity
z2nm4w0dhydj5zmowvqcxnoks
overall progress: 4 out of 4 tasks
1/4: running [=====>]
2/4: running [=====>]
3/4: running [=====>]
4/4: running [=====>]
verify: Service z2nm4w0dhydj5zmowvqcxnoks converged
```

- Verify services on node 1

```
[node1] (local) root@192.168.0.24 ~
$ docker service ls
ID                NAME        MODE           REPLICAS    IMAGE           PORTS
z2nm4w0dhydj     backend    replicated     4/4         alpine:latest
```

- View services tasks

```
[node1] (local) root@192.168.0.24 ~
$ docker service ps backend
ID                NAME        IMAGE           NODE        DESIRED STATE   CURRENT STATE           ERROR           PORTS
a3ozwlqczuxo     backend.1   alpine:latest  node2       Running          Running 2 minutes ago
2oqx14jggshs     backend.2   alpine:latest  node1       Running          Running 2 minutes ago
plakj3gi22ok     backend.3   alpine:latest  node2       Running          Running 2 minutes ago
dfkvob6saejf     backend.4   alpine:latest  node1       Running          Running 2 minutes ago
```

tasks are running on node1 and node 2

It confirms below

```
[node1] (local) root@192.168.0.24 ~
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
b624748840bf   alpine:latest  "sleep infinity"        4 minutes ago  Up 4 minutes  backend.4.dfkvob6saejfk2xc8qo12qwry
76bb41aad2e3   alpine:latest  "sleep infinity"        4 minutes ago  Up 4 minutes  backend.2.2oqx14jggshsdpgexghoxt5o

[node2] (local) root@192.168.0.23 ~
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS          NAMES
cc927035982c   alpine:latest  "sleep infinity"        4 minutes ago  Up 4 minutes  backend.1.a3ozwlqczuxos4hez0tydpmoq
36737f0ac67b   alpine:latest  "sleep infinity"        4 minutes ago  Up 4 minutes  backend.3.plakj3gi22ok4e5umnfgxl9jz
```

Step5:

Now check appnet in node 2

```
[node2] (local) root@192.168.0.23 ~
$docker network inspect appnet
[
  {
    "Name": "appnet",
    "Id": "308yqs2lir6wbpgsbfmts2llp",
    "Created": "2025-12-17T03:35:56.760317914Z",
    "Scope": "swarm",
    "Driver": "overlay",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "10.0.1.0/24",
          "Gateway": "10.0.1.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
      "36737f0ac67b2b16d9c8684883af10a0b46a0ce28b28834ec7aa12d900aacead": {
        "Name": "backend.3.plakj3gi22ok4e5umnfqxlqjz",
        "EndpointID": "bc6dfa978d55a42eb314a2fb23d185b38bcf89df619a2f30452e2ef9aa5a5f55",
        "MacAddress": "02:42:0a:00:01:05",
        "IPv4Address": "10.0.1.5/24",
        "IPv6Address": ""
      },
      "cc927035982cb32948dc7866cbf02a1b1da25549b4fe32333e509d475927441d": {
        "Name": "backend.1.a3ozwlqcuxos4hez0tydpmoq",
        "EndpointID": "9fcd379b48620603b84ff8263df17c043504dba8ad8245075424b4295cd5d0e6",
        "MacAddress": "02:42:0a:00:01:03",
        "IPv4Address": "10.0.1.3/24",
        "IPv6Address": ""
      },
      "1b-appnet": {
        "Name": "appnet-endpoint",
        "EndpointID": "e9a8de4d30a6cb288a0f761fd3aed1ae41a638bdeae89ce7e12cdfaabf192ff",
        "MacAddress": "02:42:0a:00:01:07",
        "IPv4Address": "10.0.1.7/24",
        "IPv6Address": ""
      }
    },
    "Options": {
      "com.docker.network.driver.overlay.vxlanid_list": "4097"
    },
    "Labels": {},
    "Peers": [
      {
        "Name": "dd8b096f5b46",
        "IP": "192.168.0.23"
      },
      {
        "Name": "d08527359b3a",
        "IP": "192.168.0.24"
      }
    ]
  }
]
```

Selecting a running container

```
[node1] (local) root@192.168.0.24 ~
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
b624748840bf   alpine:latest "sleep infinity"        8 minutes ago Up 8 minutes   backend.4.dfkvob6saejfk2xc8qol2qwry
76bb41aad2e3   alpine:latest "sleep infinity"        8 minutes ago Up 8 minutes   backend.2.2oqx14jggshadpgekxghoxt5o
```

Selected and installed ping utility


```
[node1] (local) root@192.168.0.24 ~  
$ docker exec -it b624748840bf sh  
/ # apk add --no-cache iputils  
(1/6) Installing libcap2 (2.77-r0)  
(2/6) Installing iputils-arping (20250605-r0)  
(3/6) Installing iputils-clockdiff (20250605-r0)  
(4/6) Installing iputils-ping (20250605-r0)  
(5/6) Installing iputils-tracepath (20250605-r0)  
(6/6) Installing iputils (20250605-r0)  
Executing busybox-1.37.0-r29.trigger  
OK: 8 MiB in 22 packages
```

Ping node 2 from node 1

```
/ # ping -c 4 192.168.0.23  
PING 192.168.0.23 (192.168.0.23) 56(84) bytes of data.  
64 bytes from 192.168.0.23: icmp_seq=1 ttl=63 time=0.215 ms  
64 bytes from 192.168.0.23: icmp_seq=2 ttl=63 time=0.136 ms  
64 bytes from 192.168.0.23: icmp_seq=3 ttl=63 time=0.126 ms  
64 bytes from 192.168.0.23: icmp_seq=4 ttl=63 time=0.103 ms  
  
--- 192.168.0.23 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3000ms  
rtt min/avg/max/mdev = 0.103/0.145/0.215/0.042 ms
```

Step 6:

Check DNS configuration in node 1 container

```

/ # cat /etc/resolv.conf
# Generated by Docker Engine.
# This file can be edited; Docker Engine will not make further changes once it
# has been modified.

nameserver 127.0.0.11
search 51ur3jppi0eupdptvsj42kdvgc.bx.internal.cloudapp.net
options ndots:0

# Based on host file: '/etc/resolv.conf' (internal resolver)
# ExtServers: [host(127.0.0.11)]
# Overrides: []
# Option ndots from: host

```

Ping service by name (DNS resolution test)

```

/ # ping -c 5 backend
PING backend (10.0.1.2) 56(84) bytes of data.
64 bytes from 10.0.1.2: icmp_seq=1 ttl=64 time=0.124 ms
64 bytes from 10.0.1.2: icmp_seq=2 ttl=64 time=0.107 ms
64 bytes from 10.0.1.2: icmp_seq=3 ttl=64 time=0.070 ms
64 bytes from 10.0.1.2: icmp_seq=4 ttl=64 time=0.071 ms
64 bytes from 10.0.1.2: icmp_seq=5 ttl=64 time=0.072 ms

--- backend ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4000ms
rtt min/avg/max/mdev = 0.070/0.088/0.124/0.022 ms

```

The ip here is the VIP

Exit container

```

/ # exit
[node1] (local) root@192.168.0.24 ~

```

Inspecting backend service from node 1

```
[node1] (local) root@192.168.0.24 ~
$ docker service inspect backend
[
  {
    "ID": "z2nm4w0dhydjszmowvqcxnoks",
    "Version": {
      "Index": 20
    },
    "CreatedAt": "2025-12-17T03:35:56.595888463Z",
    "UpdatedAt": "2025-12-17T03:35:56.598749247Z",
    "Spec": {
      "Name": "backend",
      "Labels": {},
      "TaskTemplate": {
        "ContainerSpec": {
          "Image": "alpine:latest@sha256:51183f2cfa6320055da30872f211093f9ff1d3cf06f39a0bdb212314c5dc7375",
          "Args": [
            "sleep",
            "infinity"
          ],
          "Init": false,
          "StopGracePeriod": 10000000000,
          "DNSConfig": {},
          "Isolation": "default"
        },
        "Resources": {
          "Limits": {},
          "Reservations": {}
        },
        "RestartPolicy": {
          "Condition": "any",
          "Delay": 5000000000,
          "MaxAttempts": 0
        },
        "Placement": {
          "Platforms": [
            {
              "Architecture": "amd64",
              "OS": "linux"
            },
            {
              "Architecture": "unknown",
              "OS": "unknown"
            }
          ]
        }
      }
    }
  }
]
```

There is the vip

```
    "VirtualIPs": [
      {
        "NetworkID": "3o8yqs2lir6wbpgsbfmts2llp",
        "Addr": "10.0.1.2/24"
      }
    ]
  }
]
```

This is the same IP we got the ping response from when tried ping backend

Step7:

Cleanup

Remove services

```
[node1] (local) root@192.168.0.24 ~  
$ docker service rm backend  
backend
```

```
[node1] (local) root@192.168.0.24 ~  
$ docker ps  
CONTAINER ID    IMAGE    COMMAND    CREATED    STATUS    PORTS    NAMES  
[node2] (local) root@192.168.0.23 ~  
$ docker ps  
CONTAINER ID    IMAGE    COMMAND    CREATED    STATUS    PORTS    NAMES
```

Check overlay network from node 2

```
[node2] (local) root@192.168.0.23 ~  
$ docker network ls  
NETWORK ID      NAME                DRIVER    SCOPE  
075e23bd7409    bridge              bridge    local  
6fc1819fdc3c    docker_gwbridge     bridge    local  
3047c33dd453    host                host      local  
w8k8bc3syjxg    ingress             overlay   swarm  
a432d775a218    none                null      local
```

Remove appnet from node 1

```
[node1] (local) root@192.168.0.24 ~  
$ docker network rm appnet  
appnet
```

```
[node1] (local) root@192.168.0.24 ~  
$ docker network ls  
NETWORK ID      NAME                DRIVER    SCOPE  
9a5a80d04908    bridge              bridge    local  
57792a18a041    docker_gwbridge     bridge    local  
b767b1fb0d05    host                host      local  
w8k8bc3syjxg    ingress             overlay   swarm  
ef29292b3973    none                null      local
```